

MAANG JavaScript Interview Guide

A deep-dive technical reference on lexical environments, scope chains, and V8 engine mechanics.

FUNDAMENTALS

1. What is scope in JavaScript, and why is it necessary?

Scope refers to the current context of execution in which values and expressions are visible or can be referenced. It is necessary for security, namespace management, and memory efficiency. Without scope, every variable would exist in a single global namespace, leading to unavoidable naming collisions and making garbage collection of short-lived variables impossible.

2. What is the difference between global, function, and block scope?

Global scope encompasses variables declared outside any function or block, accessible universally. **Function scope** limits visibility of variables to the function body they are defined in. **Block scope** confines variables to the nearest enclosing `{ ... }` block (e.g., an `if` statement or `for` loop), applicable only to `let` and `const` declarations.

3. Which JavaScript declarations are function-scoped and which are block-scoped?

`var` declarations, along with traditional `function` declarations (in non-strict mode browsers), are function-scoped. Conversely, `let`, `const`, and `class` declarations are block-scoped. Strict mode modifies function declarations inside blocks to be block-scoped.

4. What is lexical scoping?

Lexical (or static) scoping means that the accessibility of variables is determined by the physical placement of variables and blocks in the author-written source code. A child function automatically has access to its parent's scope, precisely where it is defined, regardless of where or how it is later invoked.

5. Does JavaScript support dynamic scoping? Explain with an example.

No, JavaScript does strictly lexical scoping. Dynamic scoping would resolve variables based on the call stack at runtime rather than where the function was authored. However, JavaScript's `this` keyword exhibits dynamic-like behavior because its value is determined entirely by how the function is invoked at runtime. For example, calling `obj.method()` binds `this` to `obj`, but assigning `const func = obj.method; func()` resolves `this` to the global object or `undefined`.

6. What is the scope chain?

The scope chain is a fundamental mechanism the JavaScript engine uses to resolve variables. It is a linked list of Lexical Environments. When a variable is accessed, the engine looks in the current environment's Environment Record. If not found, it traverses to the outer environment via the outer environment reference, continuing until the global environment is reached.

7. How does JavaScript resolve an identifier?

Identifier resolution occurs via a traversal of the Lexical Environment chain. The engine queries the innermost Environment Record for a matching binding. If missing, it queries the `[[OuterEnv]]` reference recursively. If the chain ends at the Global Environment and the identifier remains unresolved, the engine throws a `ReferenceError` (or returns `undefined` for `typeof` checks).

8. What is the difference between scope and execution context?

Scope is a static, conceptual model dictating variable visibility based on code structure. An Execution Context (EC) is a dynamic internal runtime specification object created when code runs. An EC contains state (like the call stack state), a `this` binding, and pointers to the active `LexicalEnvironment` and `VariableEnvironment`. Scope is the *rule*; the Execution Context's environment components are the runtime *implementation* of that rule.

9. What is shadowing in JavaScript?

Shadowing occurs when a variable declared in an inner scope shares the exact same identifier as a variable in an outer scope. Because identifier resolution stops at the first match in the scope chain, the inner declaration "shadows" (hides) the outer one, making the outer variable temporarily inaccessible by its simple identifier.

10. What is the difference between var, let, and const from a scoping perspective?

`var` is function-scoped, gets hoisted, and initializes with `undefined`. `let` and `const` are block-scoped, get hoisted, but remain uninitialized in the Temporal Dead Zone (TDZ) until their definition is evaluated. Redeclaring the same identifier with `var` in the same scope is allowed; with `let` or `const` it throws a `SyntaxError`.

INTERMEDIATE

11. What is a Lexical Environment?

According to the ECMAScript specification, a Lexical Environment is a specification type used to define the association of Identifiers to specific variables and functions based upon the lexical nesting structure of ECMAScript code. It consists of two parts: an Environment Record and a possibly null reference to an outer Lexical Environment.

12. What is an Environment Record?

An Environment Record is the data structure embedded within a Lexical Environment that actually stores the identifier bindings (variable names to values) created within that specific scope. It acts like a local dictionary or hash map for that scope.

13. What is a Declarative Environment Record?

A Declarative Environment Record directly binds identifiers to values. It is used for scopes that contain `var`, `const`, `let`, `class`, `module`, and `function` declarations. It is an abstract, engine-internal structure without a direct reflection in standard JavaScript objects (unlike Object Environment Records).

14. What is an Object Environment Record?

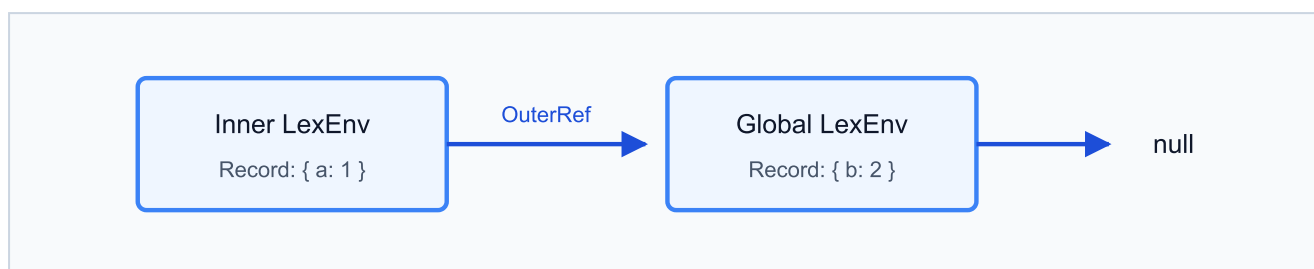
An Object Environment Record binds identifiers to the properties of a specific, tangible JavaScript object, called the binding object. This is primarily used for the global object (e.g., `window` in browsers) and the deprecated `with` statement, allowing properties of the object to be resolved as if they were standalone local variables.

15. What is a Global Environment Record?

The Global Environment Record is a composite structure combining a Declarative ER and an Object ER. Declarations like `let` and `const` at the top level go into the Declarative record (they do not become properties of the global object). Declarations like `var` and `function` go into the Object record and thus become properties of the global object (`window.x`).

16. What is the outer environment reference?

The outer environment reference (`[[OuterEnv]]`) is a pointer in a Lexical Environment that points to its parent Lexical Environment. This creates the linked list known as the scope chain. For the Global Environment, this reference is `null`.



17. How does JavaScript perform lexical lookup?

When an identifier is evaluated, the engine calls `GetIdentifierReference(lex, name, strict)`. It checks if `lex.EnvironmentRecord.HasBinding(name)` is true. If yes, it returns the reference. If no, it checks `lex.[[OuterEnv]]`. If the outer environment is `null`, it returns a reference with an undefined base (resulting in a `ReferenceError` on read).

18. What is illegal shadowing?

Illegal shadowing occurs when an inner scope attempts to declare a variable using `var` when a block-scoped variable (`let` or `const`) with the same name already exists in an enclosing block scope. Because `var` bindings hoist to the nearest enclosing function (bypassing blocks), it attempts to declare a binding where the `let` binding already uniquely occupies the identifier.

```
let x = 1;
{ var x = 2; } // SyntaxError: Identifier 'x' has already been declared
```

19. Why does `let` inside a block allow shadowing a `var` declared outside the block, but the reverse can produce an error?

A `let` inside a block stays confined to that block. It safely creates a new Declarative binding that shadows the outer `var` without interacting with the outer function scope. The reverse (inner `var`, outer `let`) fails because the inner `var` hoists entirely out of the block to the function level, colliding with the outer block's `let` declaration.

20. What is the difference between Lexical Environment and Variable Environment?

In an Execution Context, both are Lexical Environments. Historically, `VariableEnvironment` holds bindings created by `var` and `function` declarations (function-scoped), while `LexicalEnvironment` holds bindings created by `let`, `const`, `class`, and block bindings. When a new block executes, the `LexicalEnvironment` pointer is updated to a new environment, while the `VariableEnvironment` pointer remains unchanged, anchoring `var` hoisting.

21. Why should you not assume Lexical Environment and Variable Environment are literal objects inside V8?

The ECMAScript specification explicitly states these environments are "purely specification mechanisms and need not correspond to any specific artefact of an ECMAScript implementation." Engines like V8 compile variables to physical memory offsets (registers or stack slots) using static analysis during bytecode generation, avoiding the overhead of creating and hashing runtime objects entirely, except in edge cases like `eval`.

22. How does module scope differ from global scope?

Variables declared at the top level of an ES Module are isolated within a Module Environment Record. They are not added to the global object, and they are invisible to other scripts or modules unless explicitly exported. In contrast, global scope variables are shared across the entire environment.

23. What happens to top-level `var` in a browser classic script?

In a classic script (not a module), a top-level `var` declaration adds a property directly to the global `window` object via the Object Environment Record component of the Global Environment. This makes it instantly vulnerable to collision with other classic scripts loaded in the page.

24. How does top-level scope differ between browser JavaScript and Node.js?

In browsers (classic scripts), top-level scope is the true global scope (`window`). In Node.js, every file is wrapped in an invisible CommonJS module wrapper function `(function(exports, require, module, __filename, __dirname) { ... })`. Thus, "top-level" `var` or `let` inside a Node.js file is actually localized to that wrapper function's scope, not the `global` object.

ADVANCED

25. Explain closures in terms of Lexical Environments rather than simply saying "a function remembers variables."

A closure is a function combined with its lexical environment. When a function object is instantiated, the active Lexical Environment is saved into the function's internal `[[Environment]]` slot. When the function is later executed, a new Lexical Environment is created for that execution, and its outer reference (`[[OuterEnv]]`) is set to the saved `[[Environment]]`. This guarantees resolution of bindings relative to the function's birthplace.

26. Why does a closure remain able to access a variable after the outer function has returned?

In JavaScript, memory is managed by garbage collection (GC) via reachability. Even though an Execution Context is popped off the call stack when an outer function returns, the Lexical Environment object created for that invocation is still referenced by the `[[Environment]]` slot of the inner function (the closure). As long as the closure is kept alive (e.g., attached to a DOM event or a timer), the associated Lexical Environment remains reachable and bypasses GC.

27. How can closures cause unexpected memory retention?

Because closures retain a reference to their entire outer Lexical Environment, any large objects or unneeded variables existing in that scope are kept alive as long as the closure exists. V8 attempts to optimize this by only capturing closed-over variables into a heap-allocated "Context" object, but complex chains or `eval` usage can thwart this, holding entire scopes hostage in memory.

28. Explain the `var` versus `let` behavior in asynchronous for loops.

`var i` is function-scoped. In a loop with asynchronous callbacks, all iterations share the same single `i` binding. By the time callbacks fire, the loop has completed, and `i` evaluates to its final value. `let i` creates a fresh, separate Declarative Environment Record for every iteration. Each callback captures a closure over a uniquely bound Lexical Environment, preserving the specific iteration's value.

```
for (let i = 0; i < 3; i++) setTimeout(() => console.log(i), 0); // 0, 1, 2
for (var j = 0; j < 3; j++) setTimeout(() => console.log(j), 0); // 3, 3, 3
```

29. Why does lexical scope depend on where a function is defined rather than where it is called?

Because the bindings of identifiers to scope records are resolved strictly during the parsing phase (compile-time) before execution. The engine wires up the scope chain hierarchically via the AST (Abstract Syntax Tree). Function invocation creates dynamic Call Stack entries, but the Scope Chain is traversed according to static, physical program layout.

30. How are closures related to the `[[Environment]]` internal slot of functions?

The `[[Environment]]` internal slot is the exact mechanical implementation of a closure in the ECMAScript spec. When a function is created, the engine captures the currently running Execution Context's Lexical Environment and stores it in `[[Environment]]`. It acts as the immutable anchor for the scope chain whenever that function is executed.

31. How does block scope interact with for loops?

A block-scoped `for` loop actually creates two layers of environment. First, a loop environment holding the loop variable (`let i`). Second, a block environment for the loop body `{ ... }`. During initialization and the update steps, the loop acts as if it creates a new binding for `i` on each iteration, copying the value from the previous iteration's environment.

32. Why can a closure observe different let values across iterations?

The ECMAScript specification explicitly defines `CreatePerIterationEnvironment` for loops using `let` or `const`. On every iteration, a brand-new Declarative Environment is generated, inheriting from the outer scope. The current value of the loop variable is copied into this new environment. When a closure is created inside the body, its `[[Environment]]` slot captures this specific iteration's unique environment record.

33. How does module scope provide isolation between JavaScript modules?

When the engine instantiates an ES Module, it creates a unique `Module Environment Record`. This environment acts as the topmost layer of the scope chain for the module, positioned just below the Global Environment. Because each module script gets its own exclusive Module Environment Record, their top-level declarations are isolated.

34. What are live bindings in ES modules, and how do they differ from copied values?

CommonJS modules export copies of values. ES Modules export *live bindings*. The exporting module and importing module share references to the exact same memory locations in a shared Environment Record. If the exporting module updates a variable internally, the importing module observes the change immediately, although the importer cannot reassign it (it behaves like a `const` reference for the importer).

DEEP-DIVE

35. Explain the relationship between Execution Context, Lexical Environment, Environment Record, and Scope Chain.

An Execution Context dictates the runtime state. It holds pointers to the active Lexical Environment. The Lexical Environment is a wrapper structure containing an Environment Record (which holds the physical key-value map of variables in that scope) and an `[[OuterEnv]]` reference. The Scope Chain is not a discrete structure; it is the logical linked list formed by traversing these `[[OuterEnv]]` pointers from the innermost Lexical Environment outward to the Global Environment.

36. Explain the specification-level difference between LexicalEnvironment and VariableEnvironment.

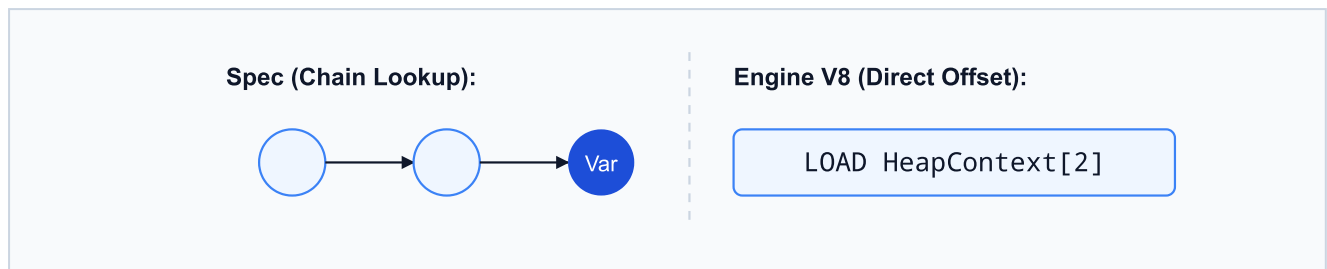
Inside an Execution Context, both are pointers to Lexical Environment structures. `VariableEnvironment` only ever updates when a new execution context starts (function entry). `LexicalEnvironment` is dynamic: it updates to point to a new, nested Lexical Environment every time block-scoped code (`{ ... }`, `catch` blocks) is entered, ensuring `let/const` isolate to the block, while `var` hoists to the stable `VariableEnvironment`.

37. How does the Global Environment Record combine object-backed and declarative bindings?

The Global Environment Record is distinct because it internally holds two separate records: an Object Record (tied to the global object) and a Declarative Record. When binding a `var`, it routes to the Object Record. When binding a `let` or `class`, it routes to the Declarative Record. The `HasBinding` and `GetBindingValue` methods seamlessly search the Declarative Record first, and if missing, fall back to the Object Record.

38. How can a JavaScript engine optimize lexical scope lookup even though the specification describes environment chains?

The specification describes dynamic string lookups across a chain. Modern engines (V8, SpiderMonkey) optimize this heavily. Because lexical scope is static, the compiler knows exactly how many scope hops are required to reach a variable. It calculates the memory offset at compile time. At runtime, instead of traversing objects, the compiled machine code accesses the exact array index on the stack or context heap object directly (e.g., `Context[2]`).



39. Why is the scope chain not the same thing as the call stack?

The call stack records the dynamic sequence of function *calls*—how control flow entered functions. The scope chain records the static sequence of function *definitions*—where variables physically live. A function may be deeply nested in the call stack but resolved against the Global Environment in the scope chain if it was defined at the top level.

40. Given a deeply nested closure, determine which bindings remain reachable and explain the potential memory consequences.

If `A` contains `B`, and `B` contains `C`, returning `C` to the global scope creates a deeply nested closure. While the specification suggests `C` retains all of `B` and `A`, V8 performs "Scope Flattening". V8 scans for which variables are actually referenced by inner functions and moves only those variables into a shared Heap Context. However, if *any* closure inside `B` or `C` uses `eval()`, V8 disables this optimization and retains the entirety of `A` and `B`'s variables, causing catastrophic memory retention.